

TP — Pentest interne de anonymous_eshop

Table of Contents

1. Stack ciblée (pour mémoire)	1
2. Préparation	2
3. Phase 1 — Cartographie	2
4. Phase 2 — Authentification et gestion de session	2
5. Phase 3 — Contrôle d'accès entre rôles (IDOR / BFLA)	3
6. Phase 4 — Logique métier et paiement	3
7. Phase 5 — Upload de fichiers	3
8. Phase 6 — Injections et validation des entrées	4
9. Phase 7 — Configuration et exposition	4
10. Rapport	4

Ce TP cible **votre propre application** `anonymous_eshop` (marketplace multi-rôles : acheteur / vendeur / admin), en environnement de développement local. C'est un exercice de pentest "boîte grise" : vous connaissez la stack, l'objectif est d'appliquer une méthodologie structurée plutôt que de découvrir une cible inconnue.

CAUTION

Tests uniquement en local (`localhost`), jamais contre le domaine de production `anonymous.shop` ni avec des clés Stripe `sk_live_*/pk_live_*`. Utilisez un `.env` local avec des clés Stripe `sk_test_*/pk_test_*`.

1. Stack ciblée (pour mémoire)

Composant	Détail
Backend	Spring Boot 3.3.4, port <code>8080</code> , JWT (JJWT 0.12.6) + TOTP 2FA, Spring Security
Base de données	PostgreSQL 16 (Flyway), Redis 7 (cache, panier, compteurs brute-force, révocation JWT)
Frontends	3 SPA React/Vite : buyer-portal (<code>:5173</code>), vendor-backoffice (<code>:5174</code>), admin-console (<code>:5175</code>)
Paieement	Stripe (SDK Java 26.7.0)
Email	SendGrid (clé stockée chiffrée en base, configurée via admin console)
Upload	Images produits, annonces, logos email

2. Préparation

1. Démarrer `db` et `redis` : `docker compose -f docker-compose.yml up -d db redis` (adapter si un overlay dev existe).
2. Démarrer le backend : `mvn spring-boot:run` (dans `solution/backend`) avec un `.env` local (clés Stripe test, secrets générés par `scripts/generate-secrets.sh`).
3. Démarrer les 3 SPA en dev (voir `solution/README.md`), ou utiliser le conteneur `frontend` combiné.
4. Créer trois comptes de test : un acheteur, un vendeur, et noter l'accès admin existant (ou le process de création d'un admin).

3. Phase 1 — Cartographie

1. Parcourir les 3 SPA avec Burp en interception passive (voir `burp/03-proxy-interception.md`) pour peupler la Site map.
2. Faute de spec exposée, reconstruire la liste des endpoints à partir du trafic capturé : parcourir chaque SPA avec chaque rôle, compléter avec les appels déclenchés par les actions (ajout panier, commande, upload...), et extraire les chemins d'API référencés dans les bundles JS (recherche texte sur `/api/`). Noter pour chaque endpoint la méthode HTTP et le rôle qui semble attendu d'après son chemin et le portail qui l'appelle.
3. Se connecter avec chaque rôle et envoyer un token JWT de chacun vers **Decoder** pour lire le payload (claims de rôle, expiration, `sub`).

TIP

Livrable — un tableau endpoint → rôle attendu → rôle(s) réellement testé(s), base pour la phase 3.

4. Phase 2 — Authentification et gestion de session

1. **Activation de compte** : inspecter le lien d'activation envoyé (vendeur/admin ont des URLs d'activation dédiées — `APP_VENDOR_ACTIVATION_BASE_URL`, `APP_ADMIN_ACTIVATION_BASE_URL`). Le token est-il prévisible, réutilisable après activation, ou valide indéfiniment ?
2. **Réinitialisation de mot de passe** : déclencher un reset sur un email existant puis sur un email inexistant — comparer les réponses (temps de réponse, message) pour détecter une énumération d'utilisateurs. Tester la réutilisation du token après un premier reset réussi.
3. **TOTP (2FA)** : après avoir saisi login/mot de passe mais *avant* de valider le code TOTP, essayer d'utiliser le token/session émis à cette étape intermédiaire directement sur un endpoint protégé — le 2FA est-il réellement bloquant côté backend, ou seulement côté UI ?
4. **Brute-force** : tester le compteur Redis de brute-force sur le login — se verrouille-t-il par compte, par IP, ou les deux ? Essayer de le contourner en modifiant l'en-tête `X-Forwarded-For` entre chaque tentative (Intruder).
5. **JWT** : dans Repeater, tester :

- a. passage de l'algorithme à **none** ou altération de la signature,
- b. modification du claim de rôle (**buyer** → **admin**) en resignant si l'algo et le secret sont devinables, ou en vérifiant si le backend valide réellement la signature,
- c. réutilisation d'un JWT après logout (la révocation Redis fonctionne-t-elle vraiment, ou le token reste-t-il valide jusqu'à expiration naturelle ?).

5. Phase 3 — Contrôle d'accès entre rôles (IDOR / BFLA)

C'est le point le plus critique d'une architecture à 3 rôles partageant un seul backend.

1. Avec un JWT **buyer**, rejouer dans Repeater chaque endpoint identifié comme réservé **vendor** ou **admin** en phase 1. Noter tout accès réussi (200 au lieu de 401/403).
2. **IDOR horizontal** : en tant qu'acheteur A, tenter d'accéder aux commandes/panier de l'acheteur B en changeant l'ID de ressource dans l'URL.
3. **Isolation vendeur** : le vendeur A peut-il lire, modifier ou supprimer un produit/une annonce appartenant au vendeur B en changeant simplement l'ID ?
4. **Escalade admin** : avec un JWT vendor, tester les endpoints d'administration (gestion des utilisateurs, configuration SendGrid, etc.).

TIP

Livrable — compléter le tableau de la phase 1 avec le résultat réel de chaque test (OK / IDOR / élévation de privilège).

6. Phase 4 — Logique métier et paiement

1. Intercepter une commande dans Repeater et modifier le prix ou la quantité envoyés au backend avant soumission à Stripe — le serveur recalcule-t-il le total côté serveur à partir du panier stocké, ou fait-il confiance au prix envoyé par le client ?
2. Chercher l'endpoint de webhook Stripe (réception de paiement) : est-il protégé par une vérification de signature Stripe (**Stripe-Signature** header) ? Essayer de l'appeler directement avec un payload forgé marquant une commande comme payée.
3. **Race condition** : si un stock limité existe (dernier exemplaire d'un produit), envoyer plusieurs requêtes d'achat en parallèle (Intruder en mode Cluster bomb avec un seul payload, ou script) pour tester une survente.

7. Phase 5 — Upload de fichiers

Trois types d'upload existent : produits, annonces, logos email (admin).

1. Tenter d'uploader un fichier avec une extension exécutable déguisée en image (**shell.php.jpg**, double extension, type MIME falsifié dans Burp).
2. Tester un **SVG avec script embarqué** — s'il est servi tel quel et affiché dans une balise **** ou

ouvert directement, cela peut déclencher une XSS stockée.

3. Tester un nom de fichier contenant `../` (path traversal) pour voir si le fichier peut être écrit hors du répertoire d'upload prévu (`APP_UPLOAD_DIR`, `APP_PRODUCT_UPLOAD_DIR`, `APP_EMAIL_LOGO_UPLOAD_DIR`).

8. Phase 6 — Injections et validation des entrées

1. Sur les champs de recherche/filtre/tri des produits, tester des caractères spéciaux SQL (`'`, `--`) — peu probable avec Hibernate/JPQL paramétré, mais vérifier les requêtes natives éventuelles ou les paramètres de tri dynamiques.
2. Sur les champs texte visibles par d'autres utilisateurs (description produit, annonce vendeur), injecter `<script>alert(document.domain)</script>` et vérifier si le rendu React échappe correctement la sortie (XSS stockée).
3. Si l'intégration La Poste ou l'envoi d'email accepte une URL fournie par l'utilisateur, tester une SSRF basique (URL vers <http://169.254.169.254> ou un service interne).

9. Phase 7 — Configuration et exposition

1. Vérifier la configuration CORS (`APP_ALLOWED_ORIGINS`) : en environnement proche de la prod, un `Origin` arbitraire est-il reflété dans `Access-Control-Allow-Origin` ?
2. Provoquer des erreurs serveur (entrée invalide, ID inexistant) et vérifier qu'aucune stack trace Java ni requête SQL ne fuite dans la réponse.
3. Vérifier le bundle JS des 3 SPA (Decoder/recherche texte) pour s'assurer qu'aucun secret (clé Stripe secrète, clé SendGrid, JWT secret) n'y est présent — seule la clé Stripe **publique** doit apparaître côté client.

10. Rapport

Consigner chaque constat avec le template `bugbounty/methodologie-bugbounty.adoc` (sections 6 à 8) : catégorie, endpoint, preuve de concept (requête Repeater exportée), sévérité, recommandation.